TokenPayload

`class` authx.schema.TokenPayload

Bases: `BaseModel`

A comprehensive Pydantic model for managing JSON Web Token (JWT) payloads with advanced token lifecycle handling.

Provides robust functionality for creating, validating, and manipulating authentication tokens with flexible configuration and extensive metadata support.

ATTRIBUTE	DESCRIPTION
`jti`	Unique token identifier. **TYPE:** `Optional[str]`
`iss`	Token issuer. **TYPE:** `Optional[str]`
`sub`	Subject (user) identifier. **TYPE:** `str`
`aud`	Token audience. **TYPE:** `Optional[StringOrSequence]`
`exp`	Token expiration time. **TYPE:** `Optional[DateTimeExpression]`
`nbf`	Token not-before time. **TYPE:** `Optional[Union[Numeric, DateTimeExpression]]`
`iat`	Token issued-at time. **TYPE:** `Optional[Union[Numeric, DateTimeExpression]]`
`type`	Token type (access or refresh). **TYPE:** `Optional[str]`
`csrf`	Cross-Site Request Forgery token. **TYPE:** `Optional[str]`
`scopes`	List of token scopes.

ATTRIBUTE	DESCRIPTION
	TYPE: `Optional[list[str]]`
`fresh`	Flag indicating if the token is freshly issued. **TYPE:** `bool`

METHOD	DESCRIPTION
`issued_at`	Converts issued time to datetime.
`expiry_datetime`	Calculates token expiration datetime.
`time_until_expiry`	Calculates remaining time before token expiration.
`time_since_issued`	Calculates time elapsed since token issuance.
`has_scopes`	Checks if token has specific scopes.
`encode`	Creates a JWT token.
`decode`	Decodes and validates a JWT token.

attr **model_config** `class-attribute` `instance-attribute`

```
model_config = ConfigDict(extra='allow', from_attributes=True)
```

attr **jti** `class-attribute` `instance-attribute`

```
jti = Field(default_factory=get_uuid)
```

attr **iss** `class-attribute` `instance-attribute`

```
iss = None
```

attr **sub** `instance-attribute`

```
sub
```

attr **aud** `class-attribute` `instance-attribute`

```
aud = None
```

attr **exp** `class-attribute` `instance-attribute`

```
exp = None
```

attr **nbf** `class-attribute` `instance-attribute`

```
nbf = None
```

attr **iat** `class-attribute` `instance-attribute`

```
iat = Field(default_factory=lambda: int(get_now_ts()))
```

attr **type** `class-attribute` `instance-attribute`

```
type = Field(default='access', description='Token type')
```

attr **csrf** `class-attribute` `instance-attribute`

```
csrf = ''
```

attr **scopes** `class-attribute` `instance-attribute`

```
scopes = None
```

attr **fresh** `class-attribute` `instance-attribute`

```
fresh = False
```

attr **extra_dict** `property`

```
extra_dict
```

Retrieve additional fields not defined in the base Pydantic model schema.

RETURNS	DESCRIPTION
`dict[str, Any]`	A dictionary containing additional fields beyond the model's defined schema.

attr **issued_at** `property`

```
issued_at
```

Convert the token's issued-at timestamp to a datetime object.

Transforms the issued-at (iat) claim into a standardized UTC datetime representation.

RETURNS	DESCRIPTION
`datetime`	A datetime object representing the token's issuance time.

RAISES	DESCRIPTION
`TypeError`	If the issued-at claim is not a float, int, or datetime object.

attr **expiry_datetime** `property`

```
expiry_datetime
```

Convert the token's expiration claim to a precise datetime object.

Transforms the expiration (exp) claim into a standardized UTC datetime representation, supporting multiple input types.

RETURNS	DESCRIPTION
`datetime`	A datetime object representing the token's expiration time.

RAISES	DESCRIPTION
`TypeError`	If the expiration claim is not a float, int, datetime, or timedelta object.

`attr` time_until_expiry `property`

```
time_until_expiry
```

Calculate the remaining time before the token expires.

Computes the time difference between the token's expiration datetime and the current time.

RETURNS	DESCRIPTION
`timedelta`	A timedelta object representing the remaining time until token expiration.

`attr` time_since_issued `property`

```
time_since_issued
```

Calculate the elapsed time since the token was issued.

Computes the time difference between the current time and the token's issued datetime.

RETURNS	DESCRIPTION
`timedelta`	A timedelta object representing the time elapsed since token issuance.

`meth` has_scopes

```
has_scopes(*scopes, all_required=True)
```

Check if the token contains the specified scopes.

Verifies whether the token's scopes satisfy the required scope values. Supports wildcard matching where a scope ending with ":" *matches any scope under that namespace (e.g., "admin:"* matches "admin:users").

PARAMETER	DESCRIPTION
`*scopes`	Variable number of scope strings to check against the token's scopes. **TYPE:** `str` **DEFAULT:** `()`
`all_required`	If True (default), all scopes must be present (AND logic). If False, at least one scope must be present (OR logic). **TYPE:** `bool` **DEFAULT:** `True`

RETURNS	DESCRIPTION
`bool`	A boolean indicating whether the scope requirements are satisfied.

Examples:

```
>>> payload.scopes = ["users:read", "posts:write"]
>>> payload.has_scopes("users:read")
True
>>> payload.has_scopes("users:read", "posts:write")
True
>>> payload.has_scopes("admin")
False
```

WITH WILDCARD SCOPES IN TOKEN

```
>>> payload.scopes = ["admin:*"]
>>> payload.has_scopes("admin:users")
True
>>> payload.has_scopes("admin:settings")
True
```

OR LOGIC

```
>>> payload.scopes = ["read"]
>>> payload.has_scopes("read", "admin", all_required=False)
True
```

Source code in `authx/schema.py`

```python
def has_scopes(self, *scopes: str, all_required: bool = True) -> bool:
    """Check if the token contains the specified scopes.

    Verifies whether the token's scopes satisfy the required scope values.
    Supports wildcard matching where a scope ending with ":*" matches
    any scope under that namespace (e.g., "admin:*" matches "admin:users").

    Args:
        *scopes: Variable number of scope strings to check against the token's scopes.
        all_required: If True (default), all scopes must be present (AND logic).
                      If False, at least one scope must be present (OR logic).

    Returns:
        A boolean indicating whether the scope requirements are satisfied.

    Examples:
        >>> payload.scopes = ["users:read", "posts:write"]
        >>> payload.has_scopes("users:read")
        True
        >>> payload.has_scopes("users:read", "posts:write")
        True
        >>> payload.has_scopes("admin")
        False

        # With wildcard scopes in token
        >>> payload.scopes = ["admin:*"]
        >>> payload.has_scopes("admin:users")
        True
        >>> payload.has_scopes("admin:settings")
        True

        # OR logic
        >>> payload.scopes = ["read"]
        >>> payload.has_scopes("read", "admin", all_required=False)
        True
    """
    return has_required_scopes(list(scopes), self.scopes, all_required=all_required)
```

`meth` encode

```python
encode(key, algorithm='HS256', ignore_errors=True, headers=None, data=None)
```

Generate a JSON Web Token (JWT) with the current payload's claims and configuration.

Creates a signed token using the specified cryptographic key and algorithm, incorporating all token metadata.

PARAMETER	DESCRIPTION
`key`	The cryptographic key used for token signing. **TYPE:** `str`
`algorithm`	The cryptographic algorithm for token signing. Defaults to HS256. **TYPE:** `AlgorithmType` **DEFAULT:** `'HS256'`
`ignore_errors`	Flag to suppress potential encoding errors. Defaults to True. **TYPE:** `bool` **DEFAULT:** `True`

PARAMETER	DESCRIPTION
`headers`	Optional custom headers to include in the token. **TYPE:** `Optional[dict[str, Any]]` **DEFAULT:** `None`
`data`	Optional additional data to embed in the token. **TYPE:** `Optional[dict[str, Any]]` **DEFAULT:** `None`

RETURNS	DESCRIPTION
`str`	A string representing the encoded and signed JWT.

Source code in `authx/schema.py`

```python
def encode(
    self,
    key: str,
    algorithm: AlgorithmType = "HS256",
    ignore_errors: bool = True,
    headers: Optional[dict[str, Any]] = None,
    data: Optional[dict[str, Any]] = None,
) -> str:
    """Generate a JSON Web Token (JWT) with the current payload's claims and configuration.

    Creates a signed token using the specified cryptographic key and algorithm, incorporating all
token metadata.

    Args:
        key: The cryptographic key used for token signing.
        algorithm: The cryptographic algorithm for token signing. Defaults to HS256.
        ignore_errors: Flag to suppress potential encoding errors. Defaults to True.
        headers: Optional custom headers to include in the token.
        data: Optional additional data to embed in the token.

    Returns:
        A string representing the encoded and signed JWT.
    """
    # Include scopes in data if present
    token_data = data.copy() if data else {}
    if self.scopes is not None:
        token_data["scopes"] = self.scopes

    return create_token(
        key=key,
        algorithm=algorithm,
        uid=str(self.sub),
        jti=self.jti,
        issued=self.iat,
        # TODO: Fix type hinting for `type` Field
        # it's caused because Type is a string & what we expect is a TokenType
        # TokenType = Literal["access", "refresh"]
        # Investigate if it's possible to fix this
        type=self.type,  # type: ignore
        expiry=self.exp,
        fresh=self.fresh,
        csrf=self.csrf,
        audience=self.aud,
        issuer=self.iss,
        not_before=self.nbf,
        ignore_errors=ignore_errors,
        headers=headers,
        data=token_data if token_data else None,
    )
```

`meth` **decode** `classmethod`

```
decode(token, key, algorithms=None, audience=None, issuer=None, verify=True)
```

Decode and validate a JSON Web Token (JWT) into a TokenPayload instance.

Converts a signed token into a structured payload object, with optional verification and validation parameters.

PARAMETER	DESCRIPTION
`token`	The encoded JWT string to be decoded. **TYPE:** `str`
`key`	The cryptographic key used for token verification. **TYPE:** `str`
`algorithms`	Optional list of allowed cryptographic algorithms. Defaults to HS256. **TYPE:** `Optional[Sequence[AlgorithmType]]` **DEFAULT:** `None`
`audience`	Optional expected token audience. **TYPE:** `Optional[StringOrSequence]` **DEFAULT:** `None`
`issuer`	Optional expected token issuer. **TYPE:** `Optional[str]` **DEFAULT:** `None`
`verify`	Flag to enable or disable token verification. Defaults to True. **TYPE:** `bool` **DEFAULT:** `True`

RETURNS	DESCRIPTION
`TokenPayload`	A TokenPayload instance representing the decoded token's claims.

Source code in `authx/schema.py`

```python
@classmethod
def decode(
    cls,
    token: str,
    key: str,
    algorithms: Optional[Sequence[AlgorithmType]] = None,
    audience: Optional[StringOrSequence] = None,
    issuer: Optional[str] = None,
    verify: bool = True,
) -> "TokenPayload":
    """Decode and validate a JSON Web Token (JWT) into a TokenPayload instance.

    Converts a signed token into a structured payload object, with optional verification and
validation parameters.

    Args:
        token: The encoded JWT string to be decoded.
        key: The cryptographic key used for token verification.
        algorithms: Optional list of allowed cryptographic algorithms. Defaults to HS256.
        audience: Optional expected token audience.
        issuer: Optional expected token issuer.
        verify: Flag to enable or disable token verification. Defaults to True.

    Returns:
        A TokenPayload instance representing the decoded token's claims.
    """
    if algorithms is None:  # pragma: no cover
        algorithms = ["HS256"]  # pragma: no cover
    payload = decode_token(
        token=token,
        key=key,
        algorithms=algorithms,
        audience=audience,
        issuer=issuer,
        verify=verify,
    )
    return cls.model_validate(payload)
```

March 20, 2026 March 19, 2026 GitHub